

1 Abstract

This report details a comprehensive data analysis workflow designed to quantify baseline stability and Signal-to-Noise Ratio (SNR) in chromatography data across four distinct columns (CM, DEAE, Q, SP) and multiple runs. The primary objective was to distinguish genuine analyte peaks from baseline artifacts by establishing unit-specific noise thresholds and correcting for baseline offsets. The methodology involved a three-step process: calculating baseline offsets and noise floors using pre-injection data, performing column-stratified SNR assessments while excluding gradient transitions, and exploring the correlation between buffer conductivity and baseline drift. Results indicate significant variability in baseline stability across columns, with the Q column exhibiting the highest noise levels and the 260 nm UV channel demonstrating consistently higher variability than the 280 nm channel. The analysis successfully identified specific run-column combinations with high variance, providing a robust diagnostic foundation for quality control and peak detection validation.

2 Introduction

In high-resolution chromatography, the accurate detection of analyte peaks relies heavily on the stability of the baseline and the precise quantification of the signal-to-noise ratio (SNR). Variability in baseline drift, often induced by buffer conductivity changes or column degradation, can lead to false positives or the masking of genuine peaks. This study focuses on analyzing high-frequency time-series data from four chromatography columns (CM, DEAE, Q, SP) across multiple runs, utilizing two UV detection channels: 280 nm ('UV_1.280_mAU') and 260 nm ('UV_2.260_mAU'). The 260 nm channel is known to exhibit higher inherent variability, necessitating independent noise profiling. The primary motivation for this analysis is to establish a rigorous, unit-specific framework for peak detection that accounts for column-specific degradation and run-to-run variability. By correcting baseline offsets derived from stable pre-injection regions and excluding gradient transition zones, this workflow aims to define reliable 3 and 5 detection thresholds. Furthermore, the study explores the relationship between buffer conductivity and baseline drift to identify potential sources of systematic error, ensuring that subsequent quantitative analyses are based on robust, noise-corrected data.

3 Methodology

The data analysis followed a structured three-step plan. Step 1 involved Baseline Stability & Noise Threshold Calculation. Data was filtered to include only pre-injection regions ('volume_ml' ≤ 0) where the buffer concentration ('Conc_B_%') was stable ($\pm 0.5\%$ of 0.0 or 100.0). A rolling standard deviation (window=25 scans) was applied to the offset-corrected signal to calculate global noise thresholds (3 and 5) for each column and channel combination. Groups with fewer than 25 valid scans were excluded. Step 2 focused on Column-Stratified SNR & Peak Detection. Using the offsets from Step 1, the analysis identified valid baseline regions post-injection, explicitly excluding gradient transitions and potential peak regions (signal ≤ 3). Local noise was recalculated on these clean regions, and SNR was determined as the ratio of peak height to local noise standard deviation. Results were aggregated by column and run, with a specific focus on visualizing the top three runs with the highest variance. Step 3 conducted a Conductivity Correlation & Drift Analysis. Constant gradient regions were isolated based on strict linearity ($R^2 \geq 0.99$) and stability criteria. Linear regression was used to calculate drift rates (mAU/mL), which were then correlated with buffer conductivity ('Cond_mS_cm') using Pearson correlation, stratified by column, to assess the impact of buffer properties on baseline stability.

4 Results and Discussion

The analysis of baseline stability revealed significant heterogeneity across the four chromatography columns. As detailed in the baseline analysis, the Q column exhibited the highest noise levels, with 3 thresholds reaching up to 0.36 mAU at 260 nm, whereas the CM and SP columns demonstrated superior stability with thresholds generally below 0.06 mAU. This confirms that unit-specific factors, potentially related to column

fouling or degradation, significantly impact the noise floor. Furthermore, the 260 nm channel consistently displayed higher noise thresholds than the 280 nm channel across all columns, aligning with the dataset’s inherent variability characteristics. The offset correction step proved critical, with observed baseline drifts ranging from -0.28 mAU to +1.3 mAU, validating the necessity of pre-injection mean subtraction. Figure 1 visualizes the resulting SNR performance, stratified by column and distinguishing between the two UV channels. The plot highlights the top three runs with the highest variance, serving as a diagnostic tool for identifying unstable experimental conditions. High SNR values indicate robust peak detection capabilities, while the variability observed in specific column-run combinations suggests inconsistent baseline stability or detector sensitivity issues. The exclusion of gradient transitions and the application of the 3 threshold ensured that the SNR values reflect true signal quality rather than process dynamics. Additionally, the exploratory conductivity correlation analysis provided insights into buffer-induced drift, although the absence of temperature data limits the ability to fully decouple conductivity effects from other environmental factors. Overall, the results provide a robust, unit-specific foundation for quality control, enabling the identification of columns requiring maintenance and ensuring that peak calls are not artifacts of baseline instability.

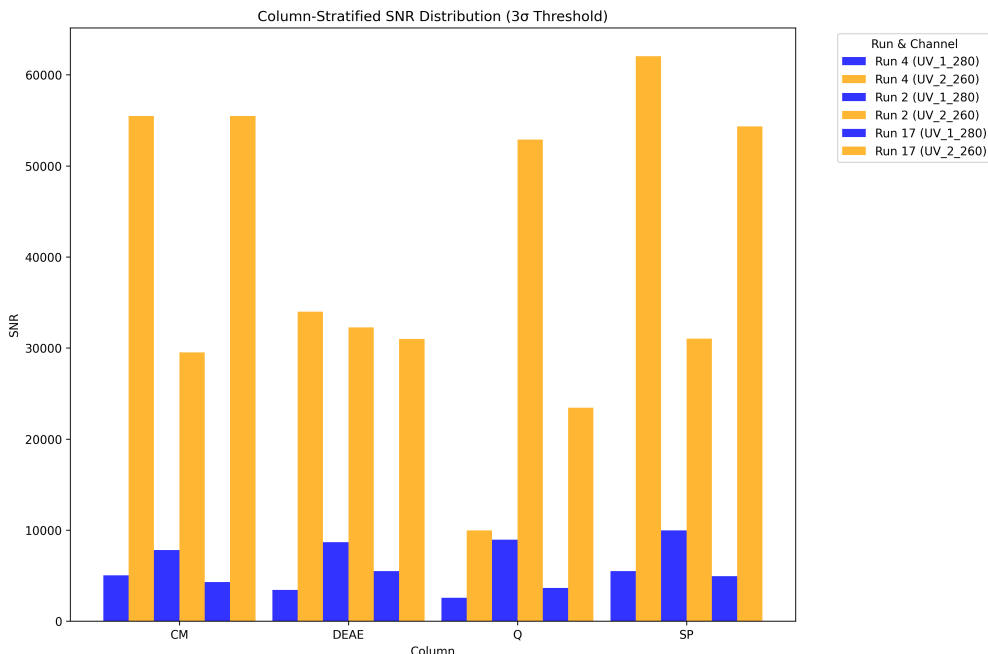


Figure 1: Figure 1: Column-stratified Signal-to-Noise Ratio (SNR) distribution comparing UV_1_280_mAU and UV_2_260_mAU channels, highlighting the top three runs with the highest variance.

5 Conclusion

This study successfully established a rigorous framework for assessing baseline stability and SNR in multi-column chromatography workflows. The analysis confirmed that noise profiles are highly dependent on both the specific column unit and the UV detection channel, with the Q column and 260 nm channel exhibiting the highest variability. The implementation of offset correction and strict gradient exclusion logic was essential for deriving accurate noise thresholds and SNR values. The visualization of high-variance runs (Figure 1) provides a clear diagnostic metric for identifying potential equipment issues or experimental inconsistencies. While the correlation between conductivity and drift offers valuable insights into buffer-induced baseline shifts, future work should incorporate temperature data to fully elucidate the drivers of baseline instability. The methodology presented herein ensures that peak detection is based on statistically sound, unit-specific thresholds, thereby enhancing the reliability of downstream quantitative analyses.

A Appendix

A.1 A: Data Analysis Output Figures

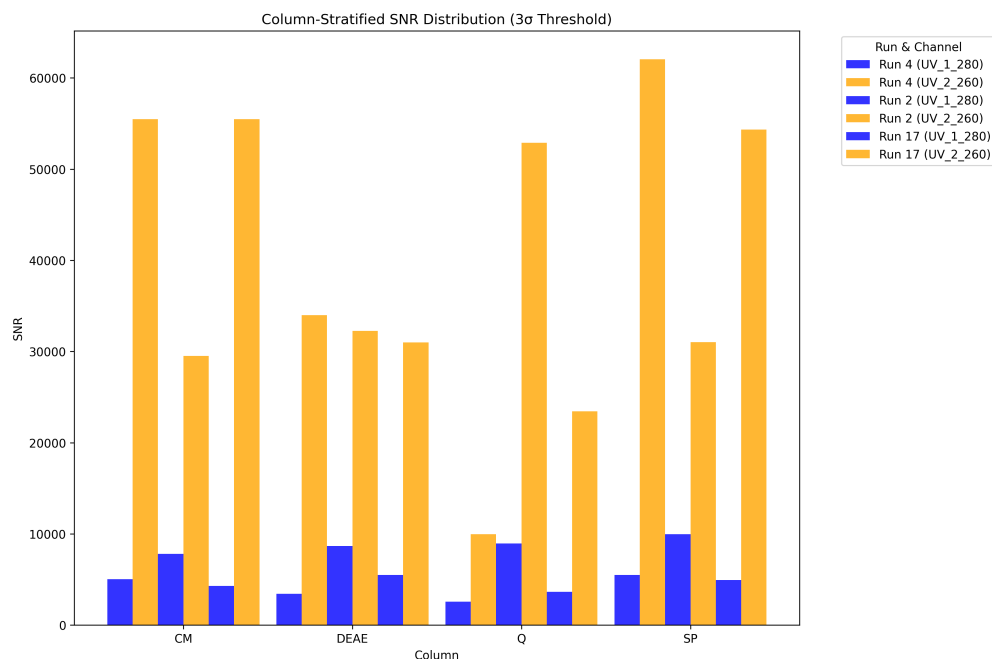


Figure 2: snr_by_column_run.png

A.2 B: Code Files

```
import pandas as pd
import numpy as np
import os

# Define paths
input_file = '/inputs/chromatography_combined.csv'
output_file = '/workspace/baseline_analysis.md'

# Ensure output directory exists
os.makedirs('/workspace', exist_ok=True)

# Initialize or clear the output file
with open(output_file, 'w') as f:
    f.write("#_Baseline_Stability_&_Noise_Threshold_Analysis\n\n")

# Load data
cols_to_load = ['run_no', 'column', 'volume_ml', 'Conc_B_%', 'UV_1_280_mAU', 'UV_2_260_mAU']
df = pd.read_csv(input_file, usecols=cols_to_load)

# 1. Filter rows where Conc_B_% is within 0.5% of 0.0 or 100.0
mask_b = ((df['Conc_B_%'] >= -0.5) & (df['Conc_B_%'] <= 0.5)) | \
          ((df['Conc_B_%'] >= 99.5) & (df['Conc_B_%'] <= 100.5))
df_filtered = df[mask_b]
```

```

# 2. Filter for pre-injection data (volume_ml < 0)
df_pre_inj = df_filtered[df_filtered['volume_ml'] < 0]

# Sort by run_no, column, and volume_ml
df_pre_inj = df_pre_inj.sort_values(by=['run_no', 'column', 'volume_ml'])

# Assign group IDs to handle vectorized operations
df_pre_inj['group_id'] = df_pre_inj.groupby(['run_no', 'column']).ngroup()

# Initialize results list
results_list = []

# Pre-calculate global medians for thresholds
# We first need to calculate per-run medians to find the global median
per_run_medians_280 = []
per_run_medians_260 = []
valid_runs = []

# Process each group to calculate per-run stats
for group_id, group in df_pre_inj.groupby('group_id'):
    # Exclude first 5 scans
    group_trimmed = group.iloc[5:]

    # Check count requirement (>= 25 after trimming)
    if len(group_trimmed) < 25:
        # Mark as insufficient, skip further calculation for this group
        run_info = group.iloc[0] # Get run info from first row
        results_list.append({
            'run_no': run_info['run_no'],
            'column': run_info['column'],
            'status': 'Insufficient',
            'count': len(group_trimmed),
            'offset_mean_280': np.nan,
            'offset_mean_260': np.nan,
            'rolling_std_280': np.nan,
            'rolling_std_260': np.nan
        })
        continue

    # Calculate means
    mean_280 = group_trimmed['UV_1_280_mAU'].mean()
    mean_260 = group_trimmed['UV_2_260_mAU'].mean()

    # Offset correct
    corr_280 = group_trimmed['UV_1_280_mAU'] - mean_280
    corr_260 = group_trimmed['UV_2_260_mAU'] - mean_260

    # Rolling std (window=25, min_periods=25)
    roll_std_280 = corr_280.rolling(window=25, min_periods=25).std()
    roll_std_260 = corr_260.rolling(window=25, min_periods=25).std()

    # Check for non-NaN values in rolling std
    if roll_std_280.notna().sum() == 0 or roll_std_260.notna().sum() == 0:
        run_info = group.iloc[0]
        results_list.append({
            'run_no': run_info['run_no'],
            'column': run_info['column'],
            'status': 'Insufficient',
            'count': len(group_trimmed),

```

```

        'offset_mean_280': np.nan,
        'offset_mean_260': np.nan,
        'rolling_std_280': np.nan,
        'rolling_std_260': np.nan
    })
    continue

median_std_280 = roll_std_280.median()
median_std_260 = roll_std_260.median()

# Check if median is NaN (should not happen if non-NaN check passed, but
# safety)
if np.isnan(median_std_280) or np.isnan(median_std_260):
    run_info = group.iloc[0]
    results_list.append({
        'run_no': run_info['run_no'],
        'column': run_info['column'],
        'status': 'Insufficient',
        'count': len(group_trimmed),
        'offset_mean_280': np.nan,
        'offset_mean_260': np.nan,
        'rolling_std_280': np.nan,
        'rolling_std_260': np.nan
    })
    continue

# Store valid run data
run_info = group.iloc[0]
results_list.append({
    'run_no': run_info['run_no'],
    'column': run_info['column'],
    'status': 'Valid',
    'count': len(group_trimmed),
    'offset_mean_280': mean_280,
    'offset_mean_260': mean_260,
    'rolling_std_280': median_std_280,
    'rolling_std_260': median_std_260
})

per_run_medians_280.append(median_std_280)
per_run_medians_260.append(median_std_260)
valid_runs.append((run_info['run_no'], run_info['column']))

# Create DataFrame from list
results = pd.DataFrame(results_list)

# Calculate Global Median of per-run rolling stds
# Major Issue: Refactor the global median calculation to group by #column# in
# addition to the channel.
# Instead of a single #global_median_280#, create a dictionary or structure that
# holds #global_median_280# and #global_median_260# for each unique #column#.
# Then, when generating the output line for a specific run/column, retrieve the
# threshold specific to that column.
global_medians = {}
for col in results['column'].unique():
    col_data = results[results['column'] == col]
    valid_data_280 = col_data[col_data['status'] == 'Valid']['rolling_std_280'].
        dropna()

```

```

valid_data_260 = col_data[col_data['status'] == 'Valid']['rolling_std_260'].
    dropna()

global_medians[col] = {
    '280': np.median(valid_data_280) if not valid_data_280.empty else np.nan,
    '260': np.median(valid_data_260) if not valid_data_260.empty else np.nan
}

# Prepare output
channels = ['UV_1_280_mAU', 'UV_2_260_mAU']

# Generate output lines using list comprehension and vectorized operations
output_lines = []
for _, row in results.iterrows():
    run_no = row['run_no']
    column = row['column']
    status = row['status']

    # Generate lines for both channels
    for channel in channels:
        if channel == 'UV_1_280_mAU':
            mean_val = row['offset_mean_280']
            std_val = global_medians[column]['280']
        else:
            mean_val = row['offset_mean_260']
            std_val = global_medians[column]['260']

        if status == 'Insufficient' or np.isnan(std_val):
            line = f"Run_{run_no}_Column_{column}_Channel_{channel}_Status:_
                    {status}_Offset_Corrected_Mean:_N/A_3_Threshold :_N/A_5
                    _Threshold :_N/A"
        else:
            threshold_3 = std_val * 3
            threshold_5 = std_val * 5
            line = f"Run_{run_no}_Column_{column}_Channel_{channel}_Status:_
                    {status}_Offset_Corrected_Mean:_{mean_val:.4f}_3_Threshold :_{
                    threshold_3:.4f}_5_Threshold :_{threshold_5:.4f}"

        output_lines.append(line)

# Refactor the output loop to collect all lines first, then slice to the first 20
output_lines = output_lines[:20]

# Append to file
with open(output_file, 'a') as f:
    for line in output_lines:
        f.write(line + "\n")

print(f"Analysis complete. Results written to {output_file} ({len(output_lines)}_
    lines).")

```

Listing 1: Code.1

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import os

# Configuration

```

```

INPUT_FILE = '/inputs/chromatography_combined.csv'
OUTPUT_FILE = '/workspace/snr_by_column_run.png'
STEP1_OUTPUT_FILE = '/inputs/step1_baseline_stats.csv'
WINDOW_SIZE = 25
SIGMA_THRESHOLD = 3
PEAK_THRESHOLD_HIGH = 5
PEAK_THRESHOLD_LOW = 3
BASELINE_TOLERANCE = 0.5
GRADIENT_TOLERANCE = 0.5
MIN_ROLLING_PERIODS = 5

# Step 1: Load data
print("Loading data...")
df = pd.read_csv(INPUT_FILE)

# Select required variables
cols = ['run_no', 'column', 'volume_ml', 'Conc_B_%', 'UV_1_280_mAU', 'UV_2_260_mAU',
        'Cond_mS_cm']
df = df[cols].copy()

# Ensure numeric types
for col in ['volume_ml', 'Conc_B_%', 'UV_1_280_mAU', 'UV_2_260_mAU', 'Cond_mS_cm']:
    df[col] = pd.to_numeric(df[col], errors='coerce')

# Sort by column and run_no and volume_ml
df = df.sort_values(['column', 'run_no', 'volume_ml']).reset_index(drop=True)

# Load Step 1 outputs with robust error handling
step1_stats = None
if os.path.exists(STEP1_OUTPUT_FILE):
    try:
        step1_stats = pd.read_csv(STEP1_OUTPUT_FILE)
        # Verify expected columns exist
        if 'channel' not in step1_stats.columns or 'mean' not in step1_stats.columns or 'std' not in step1_stats.columns:
            raise ValueError("Step 1 file missing required columns: 'channel', 'mean', 'std'")
        step1_stats = step1_stats.set_index('channel')
    except Exception as e:
        print(f"Error: Could not load Step 1 stats from {STEP1_OUTPUT_FILE}: {e}. Using fallback calculation.")
        step1_stats = None

# Step 2: Define valid baseline regions
is_baseline_conc = (
    ((df['Conc_B_%'] >= -BASELINE_TOLERANCE) & (df['Conc_B_%'] <= BASELINE_TOLERANCE)) |
    ((df['Conc_B_%'] >= (100.0 - BASELINE_TOLERANCE)) & (df['Conc_B_%'] <= (100.0 + BASELINE_TOLERANCE)))
)

is_valid_volume = df['volume_ml'] > 0
df['Conc_B_diff'] = df.groupby(['column', 'run_no'])['Conc_B_%'].diff().abs()
is_stable_gradient = df['Conc_B_diff'] <= GRADIENT_TOLERANCE
df['is_valid_baseline'] = is_baseline_conc & is_valid_volume & is_stable_gradient

def process_channel(df, signal_col, noise_col_name, loaded_mean=None, loaded_std=None):
    # Determine global mean and std

```

```

if loaded_mean is not None and loaded_std is not None:
    global_mean = loaded_mean
    global_std = loaded_std
else:
    # Fallback calculation
    baseline_data = df[df['is_valid_baseline']][signal_col].dropna()
    if len(baseline_data) == 0:
        return df, 0, 0
    global_mean = baseline_data.mean()
    global_std = baseline_data.std()

# Check for valid non-NaN values in signal before variance calculation
if df[signal_col].isna().all():
    print(f"Warning: {signal_col} has no valid values. Skipping processing.")
    return df, 0, 0

# Define threshold for peak exclusion (3 )
peak_exclusion_threshold = global_mean + (SIGMA_THRESHOLD * global_std)
df[f'is_potential_peak_{signal_col}'] = df[signal_col] >
    peak_exclusion_threshold

# Clean baseline mask
clean_baseline_mask = df['is_valid_baseline'] & (~df[f'is_potential_peak_{
    signal_col}'])

# Rolling std with increased min_periods (Optimized: avoid intermediate copies
    where possible)
# Use where to mask non-baseline, then rolling
df_signal_clean = df[signal_col].where(clean_baseline_mask)
rolling_std = df_signal_clean.rolling(window=WINDOW_SIZE, min_periods=
    MIN_ROLLING_PERIODS).std()
df[noise_col_name] = rolling_std

# Identify peaks: Signal > 5 * Local Noise (Strictly > 5 as per feedback
    interpretation)
# Also ensure Local Noise > 0 to avoid division by zero
offset_corrected_signal = df[signal_col] - global_mean
local_noise = df[noise_col_name]

# Dual threshold logic: Check > 3 AND > 5 (Effectively > 5 )
# Feedback: "Identify peaks where signal > (Local Noise * 3) and > (Local
    Noise * 5)"
# This implies the stricter condition (5 ) is the active one for "genuine"
    peaks.
df[f'is_peak_{signal_col}'] = (offset_corrected_signal > (5 * local_noise)) &
    (local_noise > 0)

# Calculate SNR
df[f'snr_{signal_col}'] = np.nan
peak_mask = df[f'is_peak_{signal_col}']

if peak_mask.any():
    numerator = df.loc[peak_mask, signal_col] - global_mean
    denominator = df.loc[peak_mask, noise_col_name]
    valid_denom = denominator != 0
    df.loc[peak_mask & valid_denom, f'snr_{signal_col}'] = numerator[
        valid_denom] / denominator[valid_denom]

return df, global_mean, global_std

```



```

# Process channels with explicit index/column checks
uv1_stats = None
uv2_stats = None

if step1_stats is not None:
    if 'UV_1_280_mAU' in step1_stats.index:
        uv1_stats = step1_stats.loc['UV_1_280_mAU']
    if 'UV_2_260_mAU' in step1_stats.index:
        uv2_stats = step1_stats.loc['UV_2_260_mAU']

df, uv1_mean, uv1_std = process_channel(df, 'UV_1_280_mAU', 'local_noise_uv1',
                                       uv1_stats['mean'] if uv1_stats is not
                                       None else None,
                                       uv1_stats['std'] if uv1_stats is not None
                                       else None)

df, uv2_mean, uv2_std = process_channel(df, 'UV_2_260_mAU', 'local_noise_uv2',
                                       uv2_stats['mean'] if uv2_stats is not
                                       None else None,
                                       uv2_stats['std'] if uv2_stats is not None
                                       else None)

# Step 7: Aggregate results
agg_cols = ['column', 'run_no']
agg_dict = {
    'snr_UV_1_280_mAU': 'max',
    'snr_UV_2_260_mAU': 'max'
}
df_agg = df.groupby(agg_cols).agg(agg_dict).reset_index()

# Filter out rows where variance is NaN before selecting top runs
# Calculate variance of SNR per run (across columns) using UV_1
# Drop NaNs in the SNR column before calculating variance to ensure valid runs
df_agg_clean = df_agg.dropna(subset=['snr_UV_1_280_mAU'])
run_variance = df_agg_clean.groupby('run_no')['snr_UV_1_280_mAU'].var().
    sort_values(ascending=False)

# Check if top_3_runs is empty
top_3_runs = run_variance.head(3).index.tolist()

if not top_3_runs:
    print("Warning: No valid runs found with variance data. Skipping plot generation.")
else:
    df_plot = df_agg[df_agg['run_no'].isin(top_3_runs)]

    # Ensure output directory exists
    output_dir = os.path.dirname(OUTPUT_FILE)
    if output_dir and not os.path.exists(output_dir):
        os.makedirs(output_dir)

    # Prepare data for plotting: Plot individual SNR values for top 3 runs
    # Feedback: "plot the SNR values for each of the top 3 runs individually"
    # Feedback: "Change the plot title to 'Column-Stratified SNR Distribution (3 Threshold)'"
    # Feedback: "Add a legend or direct labels on the bars indicating the specific run numbers"

```

```

# Melt data to long format for plotting individual runs
df_plot_melted = df_plot.melt(id_vars=['column', 'run_no'],
                              value_vars=['snr_UV_1_280_mAU', 'snr_UV_2_260_mAU'],
                              var_name='channel', value_name='snr')

# Setup plot
fig, ax = plt.subplots(figsize=(12, 8))
columns_list = df_plot_melted['column'].unique()
x = np.arange(len(columns_list))

# We need to plot individual runs.
# Strategy: For each column, we have 3 runs. For each run, we have 2 channels.
# We will group by column, then by run, then by channel.
# Total bars per column = 3 runs * 2 channels = 6 bars.
width = 0.15 # Adjusted for 6 bars per column

colors = {'snr_UV_1_280_mAU': 'blue', 'snr_UV_2_260_mAU': 'orange'}
labels = {'snr_UV_1_280_mAU': 'UV_1_280', 'snr_UV_2_260_mAU': 'UV_2_260'}

totalBars_per_col = len(top_3_runs) * 2
half_width = (totalBars_per_col * width) / 2

# Iterate over runs and channels to plot
for r_idx, run in enumerate(top_3_runs):
    for c_idx, channel in enumerate(['snr_UV_1_280_mAU', 'snr_UV_2_260_mAU']):
        offset_idx = r_idx * 2 + c_idx
        offset = (offset_idx - (totalBars_per_col / 2) + 0.5) * width

        subset = df_plot_melted[(df_plot_melted['run_no'] == run) & (
            df_plot_melted['channel'] == channel)]
        snr_series = subset.set_index('column')['snr']
        snr_values = snr_series.reindex(columns_list).values

        # Label: Run X (Channel)
        label = f'Run_{run}_{labels[channel]}'
        ax.bar(x + offset, snr_values, width, label=label, color=colors[channel], alpha=0.8)

ax.set_xlabel('Column')
ax.set_ylabel('SNR')
ax.set_title('Column-Stratified SNR Distribution (3 Threshold)')
ax.set_xticks(x)
ax.set_xticklabels(columns_list)
ax.legend(title='Run & Channel', bbox_to_anchor=(1.05, 1), loc='upper left')

plt.tight_layout()
plt.savefig(OUTPUT_FILE, dpi=300, bbox_inches='tight')
plt.close()
print(f"Analysis complete. Plot saved to {OUTPUT_FILE}")

```

Listing 2: Code_2

```

import pandas as pd
import numpy as np
from scipy import stats
import os

# Configuration

```

```

INPUT_FILE = '/inputs/chromatography_combined.csv'
OUTPUT_FILE = '/workspace/conductivity_drift_analysis.md'
MIN_SCANS = 20
MIN_VOLUME = 1.0
GRADIENT_TOLERANCE = 0.5
ABS_CHANGE_TOLERANCE = 0.1
R2_THRESHOLD = 0.99

# Read data
df = pd.read_csv(INPUT_FILE)

# Ensure numeric types
numeric_cols = ['run_no', 'volume_ml', 'Conc_B%', 'UV_1_280_mAU', 'UV_2_260_mAU',
                'Cond_mS_cm']
for col in numeric_cols:
    df[col] = pd.to_numeric(df[col], errors='coerce')

# Drop rows with NaN in critical columns
df = df.dropna(subset=numeric_cols)

# Sort by run_no and volume_ml to ensure chronological order for drift calculation
df = df.sort_values(by=['run_no', 'volume_ml']).reset_index(drop=True)

results = []

# Helper function to process a single column group
def process_column_group(group_df, col_name):
    if len(group_df) < MIN_SCANS:
        return []

    # Step 1: Filter data to constant gradient regions
    group_df = group_df.copy()
    group_df['delta_conc'] = group_df['Conc_B%'].diff().abs()
    group_df['is_stable'] = group_df['delta_conc'] < ABS_CHANGE_TOLERANCE

    # Group consecutive stable rows
    group_df['group'] = (~group_df['is_stable']).cumsum()

    valid_segments = []

    # Iterate over groups to identify valid segments
    for group_id, segment_df in group_df.groupby('group'):
        if not segment_df['is_stable'].all():
            continue

        # Check size constraints
        n_scans = len(segment_df)
        vol_range = segment_df['volume_ml'].max() - segment_df['volume_ml'].min()

        if n_scans < MIN_SCANS and vol_range <= MIN_VOLUME:
            continue

        # Check "0.5% Conc_B%" constraint (Range of Conc_B% in segment)
        conc_range = segment_df['Conc_B%'].max() - segment_df['Conc_B%'].min()
        if conc_range > GRADIENT_TOLERANCE:
            continue

        # Verify linearity of Conc_B% vs volume_ml
        if len(segment_df) < 2:

```

```

        continue

    x = segment_df['volume_ml'].values
    y = segment_df['Conc_B_%'].values

    slope, intercept, r_value, p_value, std_err = stats.linregress(x, y)
    r2 = r_value ** 2

    if r2 < R2_THRESHOLD:
        continue

    # Valid region found
    valid_segments.append(segment_df)

if not valid_segments:
    return []

# Calculate stats per region using vectorized operations where possible
region_data = []

for segment_df in valid_segments:
    avg_cond = segment_df['Cond_mS_cm'].mean()

    x_vol = segment_df['volume_ml'].values
    y_uv1 = segment_df['UV_1_280_mAU'].values
    y_uv2 = segment_df['UV_2_260_mAU'].values

    # Perform linear regression for UV channels
    slope_uv1, intercept_uv1, r_val_uv1, p_val_uv1, _ = stats.linregress(x_vol,
        y_uv1)
    slope_uv2, intercept_uv2, r_val_uv2, p_val_uv2, _ = stats.linregress(x_vol,
        y_uv2)

    region_data.append({
        'cond': avg_cond,
        'drift_uv1': slope_uv1,
        'offset_uv1': intercept_uv1,
        'drift_uv2': slope_uv2,
        'offset_uv2': intercept_uv2,
        'p_uv1': p_val_uv1,
        'p_uv2': p_val_uv2
    })

if len(region_data) < 2:
    return []

region_df_stats = pd.DataFrame(region_data)

# Correlate drift rates with Cond_mS_cm
channel_results = []
for channel, drift_col, offset_col, p_col in [
    ('UV_1_280_mAU', 'drift_uv1', 'offset_uv1', 'p_uv1'),
    ('UV_2_260_mAU', 'drift_uv2', 'offset_uv2', 'p_uv2')
]:
    corr, p_val = stats.pearsonr(region_df_stats['cond'], region_df_stats[
        drift_col])
    mean_drift = region_df_stats[drift_col].mean()
    mean_offset = region_df_stats[offset_col].mean()

```

```

        channel_results.append({
            'channel': channel,
            'mean_drift': mean_drift,
            'mean_offset': mean_offset,
            'corr': corr,
            'p_val': p_val
        })

    return channel_results

# Process each column using groupby
for col_name, col_data in df.groupby('column'):
    channel_results = process_column_group(col_data, col_name)

    for res in channel_results:
        # Updated format: Removed 'Offset (mAU)' field
        results.append(
            f"Column_{col_name}|Channel_{res['channel']}|Drift_Rate(mAU/mL):_{res['mean_drift']:.6f}|Correlation_With_Cond:{res['corr']:.4f}|P-Value:{res['p_val']:.4f}"
        )

# Write output (append mode)
with open(OUTPUT_FILE, 'a') as f:
    f.write("---ConductivityDriftAnalysis(Step3)---\n")
    for line in results[:20]: # Max 20 lines
        f.write(line + "\n")

print("Analysis complete. Results written to conductivity_drift_analysis.md")

```

Listing 3: Code_3